EDGES

OUR EDGE IS YOUR SUCCESS

Project - Smart City Delivery & Traffic Management System

Project Overview

Design and implement a comprehensive **Smart City Delivery & Traffic Management System** that simulates a real-world logistics and traffic coordination platform. The system manages package deliveries, optimizes routes, handles traffic conditions, and coordinates multiple delivery vehicles across a city network.

Core Features & Algorithm Integration

1. City Road Network (Graph)

- Represent the city as a **weighted directed graph**
  - Nodes: Intersections/Locations
  - Edges: Roads with weights (distance, traffic congestion, time)
- Implement multiple graph algorithms:
  - **Dijkstra's Algorithm**: Find shortest path between locations
  - **BFS/DFS**: Explore connectivity and reachability
  - **Kruskal's/Prim's Algorithm**: Build minimum spanning tree for infrastructure planning
  - **Bellman-Ford**: Handle negative weights (e.g., toll discounts)
  - **Topological Sort**: For one-way street planning

2. Fast Lookup System (Hashing)

- Implement custom hash tables for:
  - **Location Database**: Hash table mapping location IDs to coordinates and details
  - **Vehicle Registry**: Quick vehicle lookup by ID
  - **Package Tracking**: Track packages by tracking number
  - **Customer Database**: Store and retrieve customer information
- Use separate chaining or open addressing for collision resolution
- Implement custom hash functions for different data types

3. Spatial Indexing (Trees)

- **QuadTree** or **KD-Tree**:
  - Partition city geographically for efficient nearest-neighbor searches
  - Find all delivery points within a radius
  - Locate nearest available vehicle to a pickup location
- **Binary Search Tree (BST)**:
  - Maintain sorted list of delivery deadlines
  - Priority-based vehicle assignment
- **Segment Tree**:
  - Range queries for traffic density on road segments

4. Delivery Queue Management (Priority Queue)

- Implement priority queue (using binary heap) for:
  - **Urgent Deliveries**: Sort by deadline, priority level, distance
  - **Vehicle Assignment**: Assign vehicles based on proximity and capacity
  - **Traffic Event Handling**: Process accidents, road closures by severity
  - **Customer Service Queue**: Handle requests by priority
- Support operations: insert, extractMax/Min, updatePriority

5. Route Optimization (Greedy Algorithm)

- **Activity Selection Approach**:
  - Select deliveries that maximize profit per time unit
  - Choose optimal charging/refueling stops
- **Greedy Route Selection**:
  - At each intersection, choose next destination by minimum cost/time
  - Fractional knapsack for package loading (weight/volume constraints)

- **Resource Allocation**:
  - Assign vehicles to regions using greedy strategy
  - Minimize total travel distance/time

6. Efficient Sorting & Searching (Divide & Conquer)

- **Merge Sort**:
  - Sort deliveries by multiple criteria (location, time, priority)
  - Sort traffic data for analysis
- **Quick Sort**:
  - Fast sorting of large datasets (vehicles, packages)
- **Binary Search**:
  - Search sorted delivery schedules
  - Find optimal time slots
- **Closest Pair of Points**:
  - Find nearest delivery locations for batching
- **Divide & Conquer Route Partitioning**:
  - Split large delivery zones into smaller sub-regions
  - Recursively assign vehicles to partitions

System Components

Component 1: City Map Manager

Responsibilities:

- Load city graph from file

- Add/remove roads (edges)

- Update traffic conditions (edge weights)

- Find shortest paths between any two points

- Detect road closures and suggest alternatives

Component 2: Location & Entity Manager (Hash-based)

Responsibilities:

- Register new locations, vehicles, customers

- Fast O(1) lookup operations

- Update entity information

- Generate unique IDs with hash distribution

Component 3: Spatial Query Engine (Tree-based)

Responsibilities:

- Build QuadTree/KD-Tree from location coordinates

- Find all locations within radius

- Nearest neighbor search for vehicle assignment

- Range queries for delivery zones

Component 4: Delivery Scheduler (Priority Queue)

Responsibilities:

- Maintain priority queue of pending deliveries

- Assign deliveries to vehicles based on urgency

- Handle real-time delivery requests

- Reorder queue when priorities change

Component 5: Route Optimizer (Greedy)

Responsibilities:

- Generate optimal routes for vehicles

- Select most profitable deliveries first

- Balance load across vehicles

- Minimize empty travel distance

Component 6: Data Processing Module (Divide & Conquer)

Responsibilities:

- Sort and filter large datasets

- Partition delivery zones geographically

- Merge sorted results from different vehicles

- Binary search in sorted schedules

Key Functionalities

1. Package Delivery System

- Add new delivery requests with source, destination, deadline, priority
- Calculate optimal route using Dijkstra + Greedy selection
- Assign to nearest available vehicle (using spatial tree)
- Track delivery status (using hash table)

2. Vehicle Management

- Register vehicles with capacity, speed, fuel/battery
- Track real-time location
- Assign delivery batches (using priority queue + greedy)
- Find nearest vehicle to pickup location (using spatial tree)

3. Traffic Monitoring

- Update road conditions (construction, accidents, congestion)
- Recalculate routes dynamically
- Suggest alternative paths (graph algorithms)
- Priority-based incident handling (priority queue)

4. Multi-Delivery Route Planning

- Given N deliveries and M vehicles
- Partition deliveries geographically (divide & conquer)
- Assign clusters to vehicles (greedy assignment)
- Generate TSP-approximation routes (nearest neighbor greedy)

5. Analytics & Reporting

- Sort deliveries by completion time (merge sort)
- Find top K busiest intersections (priority queue + hash)
- Calculate average delivery time per zone
- Identify bottleneck roads in network

Data Structures Implementation Details

Graph (Adjacency List)

```
class RoadNetwork {

    unordered_map<int, vector<pair<int, double>>> adjList; // node -> [(neighbor, weight)]
```

```
    // Dijkstra, BFS, DFS, MST implementations
};
```

Hash Table (Custom)

```
template<typename K, typename V>
class HashTable {
    vector<list<pair<K, V>>> table;
    int (*hashFunc)(K);
    // insert, search, delete with collision handling
};
```

QuadTree

```
struct QuadTreeNode {
    double x, y; // center
    double width, height;
    vector<Location*> locations;
    QuadTreeNode* children[4]; // NW, NE, SW, SE
};
```

Priority Queue (Binary Heap)

```
template<typename T>
class PriorityQueue {
```

```
    vector<T> heap;

    bool (*comparator)(T, T);

    void heapifyUp(int index);

    void heapifyDown(int index);

};
```

Sample Operations Flow

Scenario: New Delivery Request

1. **Hash Table**: Look up customer and destination details
2. **Spatial Tree**: Find nearest available vehicle
3. **Priority Queue**: Insert delivery with priority based on deadline
4. **Graph**: Calculate shortest path from vehicle to pickup to destination
5. **Greedy**: If vehicle has multiple deliveries, optimize order
6. **Divide & Conquer**: Sort all pending deliveries, binary search for slot

Scenario: Rush Hour Traffic Update

1. **Graph**: Update edge weights for congested roads
2. **Hash Table**: Retrieve all active deliveries
3. **Graph**: Recalculate routes using updated weights (Dijkstra)
4. **Priority Queue**: Reorder deliveries at risk of missing deadline
5. **Greedy**: Reassign some deliveries to less busy vehicles

---

Input/Output Specifications

Input Files

- city_map.txt: Graph structure (nodes, edges, weights)
- locations.txt: Location database (id, name, coordinates, type)
- vehicles.txt: Vehicle fleet data (id, capacity, speed, start_location)
- deliveries.txt: Delivery requests (id, source, dest, deadline, priority, weight)
- traffic_updates.txt: Real-time traffic events (road_id, new_weight, timestamp)

Output

- Optimized routes for each vehicle
- Delivery schedule sorted by time
- Total distance/time metrics
- Unserved deliveries (if any)
- Traffic bottleneck analysis

Advanced Features (Optional Extensions)

1. **Multi-criteria Optimization**: Balance time, cost, fuel consumption
2. **Dynamic Rerouting**: Handle real-time obstacles
3. **Zone Partitioning**: Use divide & conquer for load balancing
4. **Predictive Analysis**: Use historical data (sorted by time) for route prediction
5. **Visualization**: Display graph, routes, vehicle positions

---

Implementation Phases

Phase 1: Foundation (Week 1)

- Implement graph class with basic operations
- Build hash table with custom hash functions

- Create basic data structures (Location, Vehicle, Delivery)

Phase 2: Core Algorithms (Week 2)

- Implement Dijkstra's algorithm
- Build QuadTree for spatial indexing
- Implement priority queue (binary heap)
- Add merge sort and binary search

Phase 3: Integration (Week 3)

- Connect components
- Implement greedy route optimizer
- Add delivery assignment logic
- Build traffic update mechanism

Phase 4: Testing & Optimization (Week 4)

- Test with large datasets
- Optimize performance
- Add edge cases handling
- Create comprehensive test suite

Complexity Requirements

- **Graph Operations**: O(E log V) for Dijkstra
- **Hash Operations**: O(1) average for insert/search
- **Spatial Queries**: O(log N) for balanced tree
- **Priority Queue**: O(log N) for insert/extract
- **Sorting**: O(N log N) for merge/quick sort
- **Binary Search**: O(log N)

Evaluation Criteria

1. **Correctness**: All algorithms implemented correctly
2. **Integration**: Seamless use of all required data structures
3. **Efficiency**: Meets complexity requirements
4. **Code Quality**: Clean, modular, well-documented
5. **Testing**: Comprehensive test cases
6. **Real-world Applicability**: Practical solution design

Deliverables

1. Source Code zipped
2. Running Executable
3. **Document with your design (The used design pattern and its mapped UML Diagram)**
4. **Video Illustrate that document**

Any missing requirement of the deliverables, your project will be considered rejected

Thank You
Edges For Training Team